0-LOOP

UNIVERSIDAD POLITÉCNICA DE PACHUCA

Proyecto: Sistema de Punto de Venta (POS) B2B - SkateShopPachuca

Materia: Aplicaciones Web Orientadas a Servicios (AWOS)

Docente: Dr.Jazmín Rodríguez Flores

Integrantes (Equipo de Alto Rendimiento):

- Angel Daniel Meneses Castilleja (Líder Frontend y Arquitectura UI, Documentación)
- Samuel de Jesús Alavez Lugo (Líder Backend Transaccional y NoSQL)
- Karel Galindo Domingo (Líder Backend Inventario y SQL)

Zempoala, Hidalgo 1 de marzo de 2026

ÍNDICE

1. **Introducción y Objetivo**
2. **Normativas y Estándares de Calidad**
3. **Metodología de Desarrollo y Herramientas**
   3.1 Stack Tecnológico
   3.2 Principios y Patrones de Diseño
4. **Arquitectura y Diseño de Base de Datos**
   4.1 Diccionario de Datos Relacional (PostgreSQL)
   4.2 Diagrama Entidad-Relación (ERD)
   4.3 Diccionario de Datos NoSQL (Firestore)
5. **Diseño de APIs REST (Contratos de Integración)**
6. **Especificación de Requerimientos y Product Backlog**
   6.1 Requerimientos Funcionales (IEEE 830)
   6.2 Requerimientos No Funcionales (RNF) - ISO/IEC 25010
7. **Historias de Usuario y Casos de Uso (Product Backlog)**
   7.1 Historias de Usuario
   7.2 Especificación de Casos de Uso (UML)
8. **Estrategia Integral de Pruebas (Estándar IEEE 829 Avanzado)**
   8.1 Pruebas Unitarias (Caja Blanca)
   8.2 Pruebas de Integración y API (Caja Negra)
   8.3 Pruebas de Sistema y UI (End-to-End)
   8.4 Pruebas de Seguridad en Base de Datos
9. **Métricas de Control Ágil y Definition of Done (DoD)**
10. **Enlaces y Repositorios Oficiales del Proyecto**
11. **Bibliografía y Referencias Técnicas**

1. Introducción y Objetivo

El Sistema de Punto de Venta (POS) "SkateShopPachuca" es una plataforma de software diseñada como herramienta de operación interna (B2B). A diferencia de las plataformas de comercio electrónico público, este sistema prioriza la integridad transaccional, la seguridad basada en roles (RBAC) y la resiliencia operativa.

Objetivo General: Desarrollar un sistema POS multiplataforma (Web y Móvil) bajo una arquitectura orientada a microservicios distribuidos, utilizando dos lenguajes de backend (Python y Node.js) y un modelo de persistencia híbrido (PostgreSQL y Firestore) para garantizar cero discrepancias de inventario físico.

---

2. Normativas y Estándares de Calidad

El proyecto se rige bajo los siguientes estándares internacionales:

- **ISO/IEC 25010 (Calidad del Producto de Software): Asegurando mantenibilidad mediante Clean Architecture, eficiencia con manejo reactivo de estados y seguridad mediante tokens JWT.**
- **IEEE 830 (Especificación de Requisitos): Redacción estructurada de Historias de Usuario con criterios de aceptación medibles.**
- **IEEE 829 (Documentación de Pruebas): Diseño y ejecución de Casos de Prueba estructurados previos al despliegue a producción.**

---

3. Metodología de Desarrollo y Herramientas

El ciclo de vida del software se rige bajo Scrum, con integraciones de Git Flow para el control de versiones.

3.1 Stack Tecnológico

Capa	Tecnología	Propósito
Frontend Híbrido	Flutter 3.19 (Dart)	Interfaz responsiva única (Web/Móvil) con manejo de estado BLoC.
Backend 1 (Core)	Python 3.11 + FastAPI	Microservicio de Inventario y Catálogo (Alta latencia, cálculos duros).
Backend 2 (Auth/Ventas)	Node.js 20 + Express	Microservicio de orquestación de ventas y JWT.
BD Relacional	PostgreSQL 18	Integridad referencial, ACID, almacenamiento del catálogo.
BD No Relacional	Firebase Firestore	Almacenamiento rápido de *tickets* inmutables y sesiones.

3.2 Principios y Patrones de Diseño

1. **Clean Architecture: Separación estricta en capas (Domain, Data, Presentation) en el cliente Flutter para aislar la lógica de negocio de la UI.**
2. **Singleton: Utilizado para inicializar una única instancia global de la conexión a Firebase (`FirebaseApp`), optimizando la memoria RAM en móviles.**
3. **Repository Pattern: Abstracción de las fuentes de datos para permitir la inyección de dependencias (Mocks) o la transición transparente a bases de datos locales.**
4. **BLoC (Business Logic Component): Gestión reactiva de estados en la interfaz para lograr 60 FPS fluidos sin recargas de pantalla completas.**

---

4. Arquitectura y Diseño de Base de Datos

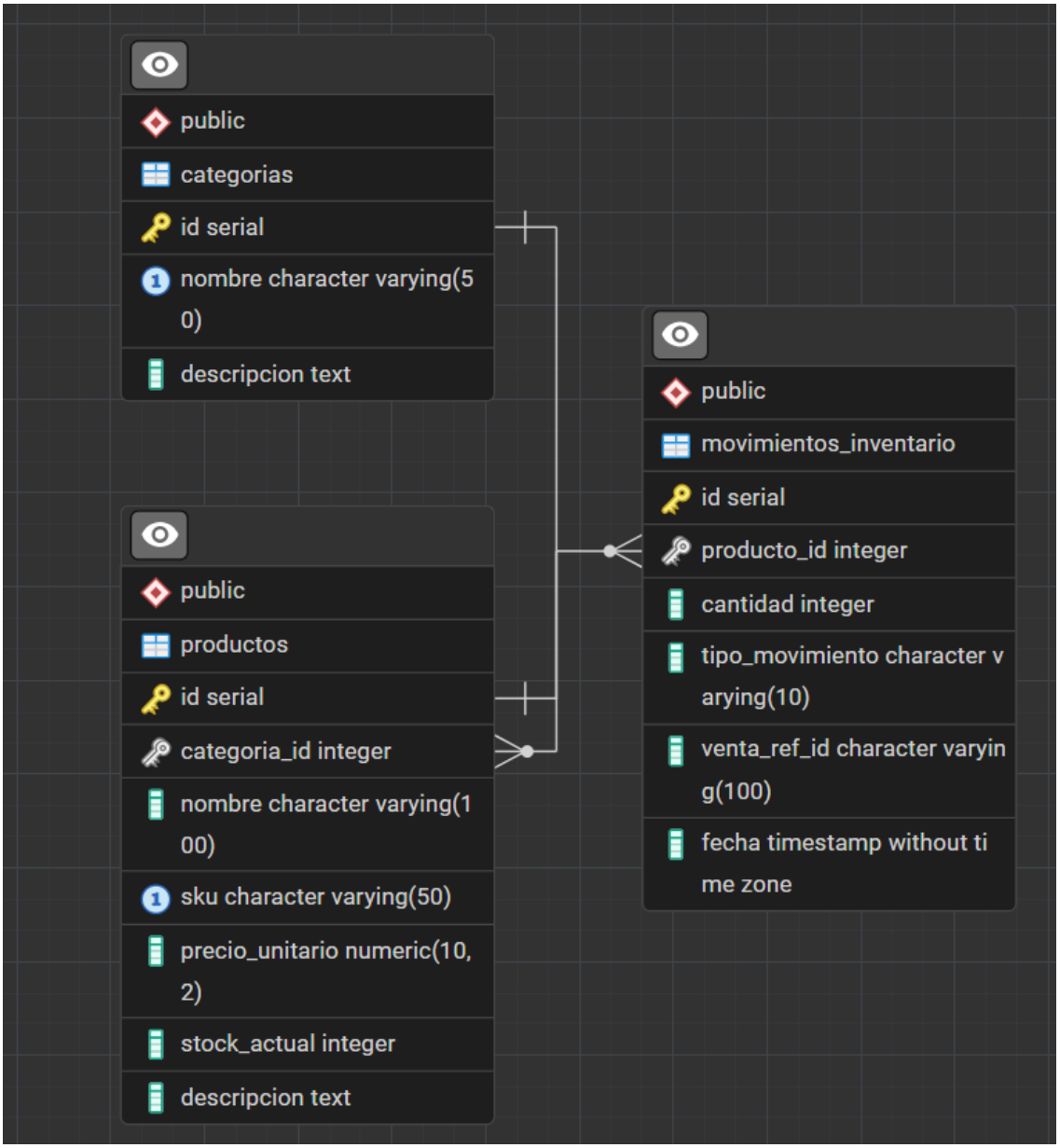
4.1 Diccionario de Datos Relacional (PostgreSQL) - Backend Python

Tabla	Campo	Tipo de Dato	Restricción	Descripción
categorias	id	SERIAL	PK	Llave primaria
	nombre	VARCHAR(100)	NOT NULL	Nombre de la categoría
productos	id	SERIAL	PK	Llave primaria
	categoria_id	INT	FK	Relación con categorías
	nombre	VARCHAR(150)	NOT NULL	Nombre del producto
	precio	DECIMAL(10,2)	NOT NULL	Precio unitario

	stock_actual	INT	NOT NULL	Existencia física (>= 0)

Trigger de Integridad: Se implementó un *Trigger* `trg_check_stock` a nivel motor de base de datos que aborta cualquier transacción donde `stock_actual < 0`, blindando el inventario contra concurrencias.

4.2 Diagrama Entidad-Relación (ERD)



4.3 Diccionario de Datos NoSQL (Firestore) - Backend Node.js

Colección `ventas` (Documento JSON Inmutable):

JSON

```
{
  "_id": "vnt_89a7f6...",
  "vendedor_id": "usr_google_uuid...",
  "timestamp": "2026-03-07T12:00:00Z",
  "metodo_pago": "efectivo",
  "total": 2400.00,
  "productos_comprados": [
    { "id_producto": 101, "nombre": "Tabla Santa Cruz", "cantidad": 2, "precio": 1200.00 }
  ]
}
```

5. Diseño de APIs REST (Contratos de Integración)

Backend	Método	Endpoint	Responsabilidad	Rol Requerido
FastAPI	GET	/api/productos	Listar catálogo activo.	Trabajador, Admin
FastAPI	POST	/api/productos/descontar	Validar y descontar stock.	Sistema Interno
Node.js	POST	/api/auth/login	Validar Google Token y Rol.	Público
Node.js	POST	/api/ventas/procesar	Orquestar el cobro y ticket.	Trabajador

6. Especificación de Requerimientos y Product Backlog

6.1 Requerimientos Estrictos (IEEE 830 / ISO 25010)

ID	Nombre	Descripción Técnica	Prioridad
RF-01	Autenticación (SSO) y Sesión	Acceso mediante Google Auth. El token JWT debe tener un tiempo de expiración de 8 horas. Tras expirar, el sistema debe forzar el cierre de sesión local por seguridad.	Crítica
RF-02	Ruteo RBAC Dinámico	Al iniciar sesión, el cliente Flutter leerá el `rol` del token. `Trabajador` -> `CatalogoPage`. `Administrador` -> `AdminDashboard`. Rutas bloqueadas por *Route Guards*.	Alta
RF-03	CRUD Catálogo Resiliente	FastAPI gestionará el catálogo en PostgreSQL. Los borrados serán lógicos (`estado = inactivo`). Los productos inactivos no aparecerán en el POS, conservando la integridad de los *tickets* históricos.	Alta
RF-04	Orquestación de Venta Distribuida	El POS envía el carrito a Node.js; Node.js ejecuta un *fetch* seguro a FastAPI (`/api/productos/descontar`). Solo si FastAPI responde `200 OK`, Node.js commitea el ticket en Firestore.	Crítica

RF-05	Rollback Transaccional	Si la venta falla en Firestore después de descontar el stock en PostgreSQL, Node.js debe enviar una petición de compensación (Rollback) a FastAPI para retornar el stock.	Alta
RF-06	Cache Local (Offline First)	El catálogo de productos se guardará en caché local (mediante Hive/Isar en Flutter) para que la búsqueda en el mostrador sea instantánea (0 ms de latencia de red).	Media
RF-07	Ajustes de Inventario	El sistema debe permitir registrar entradas y salidas manuales de mercancía, almacenando el motivo del movimiento y actualizando el stock sin generar una venta.	Alta
RF-08	Prevención Estricta de Stock	PostgreSQL debe rechazar cualquier transacción que resulte en stock negativo mediante un TRIGGER, devolviendo un código 409 Conflict hacia las APIs.	Crítica
RF-09	Anulación de Ventas	El Administrador podrá cancelar un ticket en Firestore. Esto debe detonar automáticamente una petición a FastAPI para retornar el stock de los productos cancelados.	Alta
RF-10	Auditoría y Reportes	FastAPI compilará las métricas de ventas en tiempo real y generará documentos PDF/Excel descargables utilizando la librería Pandas y ReportLab.	Alta

6.2 Requerimientos No Funcionales (RNF) - ISO/IEC 25010

ID	Atributo ISO 25010	Descripción Técnica	Validación
RNF-01	Seguridad	Todas las peticiones HTTP entre Node.js y FastAPI deben estar firmadas con un *API Key* interno (X-API-KEY) para evitar que agentes externos consuman el servicio de inventario.	Unit Testing / Postman
RNF-02	Desempeño (Rendimiento)	La UI en Flutter debe renderizar a 60 FPS. Las consultas complejas o imágenes deben usar *Skeleton Loaders* en lugar de bloquear el hilo principal de la aplicación.	Flutter DevTools
RNF-03	Tolerancia a Fallos	Si el microservicio de Python se cae, Node.js devolverá un error 503 Service Unavailable controlado al cliente móvil, mostrando un *Snackbar* amigable sin colapsar la app.	Pruebas de Estrés
RNF-04	Usabilidad (Red)	El buscador de productos en Flutter debe implementar un *Debounce* de 500ms para evitar saturar el backend con peticiones SQL por cada letra presionada por el cajero.	Pruebas de Integración

RNF-05	**Integridad de Datos**	**Las operaciones de inventario en PostgreSQL deben cumplir con las propiedades ACID, asegurando que ninguna transacción quede a medias en caso de apagón del servidor.**	**SQL Constraints**

7. Historias de Usuario y Casos de Uso (Product Backlog)

7.1: Historias de Usuario

Cada Historia de Usuario (HU) ha sido redactada asegurando trazabilidad técnica entre el frontend (Flutter), la persistencia relacional (PostgreSQL) y los microservicios distribuidos (Node.js/Python).

Módulo 1: Autenticación y Seguridad (RBAC)

HU01: Inicio de Sesión Seguro (SSO) mediante Google Auth

- **Prioridad: Crítica | Estimación: 8 Puntos de Historia**
- **Descripción: * Como empleado de SkateShopPachuca.**
    1. **Quiero iniciar sesión utilizando mi cuenta de Google.**
    2. **Para acceder a mis herramientas de trabajo de forma segura sin gestionar contraseñas locales.**
- **Criterios de Aceptación:**
    1. **La pantalla de login debe integrar el paquete `google_sign_in`.**
    2. **El token de Google debe enviarse al endpoint de Node.js (`/api/auth/login`) para validar su existencia en Firestore.**
    3. **Si el correo no está registrado, el `AuthBloc` emitirá un `AuthError` y la UI mostrará un `SnackBar` rojo: *"Acceso denegado. Contacte al administrador"*.**

HU02: Ruteo Dinámico Basado en Roles (Route Guards)

- **Prioridad: Alta | Estimación: 5 Puntos de Historia**
- **Descripción: * Como Arquitecto de Software.**
    1. **Quiero que la aplicación redirija automáticamente al usuario según su nivel de privilegios.**
    2. **Para evitar que un cajero acceda a los reportes o configuraciones del administrador.**
- **Criterios de Aceptación:**
    1. **Al recibir el estado `AuthSuccess`, el enrutador de Flutter leerá el atributo `rol`.**
    2. **`Trabajador` -> Redirige a `/pos` (Catálogo y Carrito).**
    3. **`Administrador` -> Redirige a `/dashboard` (Métricas y Gestión).**
    4. **`Consultor` -> Redirige a `/reportes`.**

HU03: Gestión y Asignación de Empleados

- **Prioridad: Alta | Estimación: 8 Puntos de Historia**
- **Descripción: * Como Administrador del sistema.**
    1. **Quiero dar de alta los correos de mis empleados y asignarles un rol.**

    2. Para autorizar quién puede operar la caja registradora.
- Criterios de Aceptación:
    1. Formulario en Flutter para ingresar email y seleccionar rol (`Trabajador`, `Administrador`, `Consultor`).
    2. Node.js verificará que el correo no esté duplicado en la colección `usuarios` de Firestore.
    3. Guardado con `timestamp` de creación y `creado_por` (ID del Admin).

HU04: Cierre de Sesión y Limpieza de Estado

- Prioridad: Media | Estimación: 3 Puntos de Historia
- Descripción: * Como usuario del sistema.
    1. Quiero cerrar mi sesión al terminar mi turno.
    2. Para proteger la integridad de la caja registradora.
- Criterios de Aceptación:
    1. Ejecutar `FirebaseAuth.instance.signOut()`.
    2. El BLoC debe emitir eventos para purgar la memoria (vaciar carrito y datos de usuario).
    3. Redirigir a `/login` eliminando el historial de navegación (`pushAndRemoveUntil`).

Módulo 2: Gestión de Inventario (FastAPI + PostgreSQL)

HU05: Alta de Catálogo de Patinetas y Accesorios

- Prioridad: Alta | Estimación: 8 Puntos de Historia
- Descripción: * Como Administrador.
    1. Quiero registrar nuevos productos especificando precio y cantidad física inicial.
    2. Para mantener la base de datos sincronizada con el almacén de la tienda.
- Criterios de Aceptación:
    1. Formulario en Flutter validando mediante Regex que el precio sea `> 0` y el stock `>= 0`.
    2. Petición `POST` a FastAPI para insertar en la tabla `productos` de PostgreSQL.
    3. Retornar mensaje de éxito y limpiar formulario.

HU06: Edición y Actualización de Precios

- Prioridad: Media | Estimación: 5 Puntos de Historia
- Descripción: * Como Administrador.
    1. Quiero modificar la información de un producto existente.
    2. Para corregir errores tipográficos o ajustar precios por inflación.
- Criterios de Aceptación:
    1. Vista de detalle con campos prellenados.

2. Petición `PUT` a FastAPI. No se debe permitir alterar el `stock_actual` desde esta vista (ver HU08).

HU07: Baja Lógica de Productos

- **Prioridad: Media | Estimación: 5 Puntos de Historia**
- **Descripción: * Como Administrador.**
  1. **Quiero desactivar productos que ya no se venderán.**
  2. **Para limpiar el catálogo sin borrar el historial de ventas pasadas.**
- **Criterios de Aceptación:**
  1. **Petición `DELETE` (o `PATCH`) a FastAPI que cambie el campo `activo = false` en PostgreSQL.**
  2. **Los productos inactivos no deben renderizarse en el catálogo del cajero, pero sí deben aparecer en reportes históricos.**

HU08: Registro de Movimientos de Inventario (Ajustes)

- **Prioridad: Alta | Estimación: 8 Puntos de Historia**
- **Descripción: * Como Administrador.**
  1. **Quiero registrar entradas (compras a proveedor) o salidas (mermas/robos).**
  2. **Para cuadrar el inventario físico con el sistema sin hacer ventas ficticias.**
- **Criterios de Aceptación:**
  1. **FastAPI insertará un registro en la tabla `movimientos_inventario` (tipo, cantidad, motivo, fecha).**
  2. **Un Trigger en PostgreSQL actualizará automáticamente el `stock_actual` de la tabla `productos`.**

HU09: Catálogo Operativo con Búsqueda Reactiva (Debounce)

- **Prioridad: Alta | Estimación: 8 Puntos de Historia**
- **Descripción: * Como Trabajador (Vendedor).**
  1. **Quiero visualizar los productos y buscar por nombre o marca rápidamente.**
  2. **Para agilizar la atención al cliente en mostrador.**
- **Criterios de Aceptación:**
  1. **Interfaz tipo Grid responsivo (SliverGrid).**
  2. **El campo de búsqueda implementará un *Debounce* de 500ms para no saturar a FastAPI con peticiones por cada tecla presionada.**
  3. **Mostrar etiqueta roja "Stock Crítico" si el producto tiene `< 5` unidades.**

HU10: Prevención de Stock Negativo a Nivel Base de Datos

- **Prioridad: Crítica | Estimación: 5 Puntos de Historia**
- **Descripción: * Como Sistema / Arquitecto.**
  1. **Quiero delegar la validación de inventario al motor PostgreSQL.**
  2. **Para garantizar que ninguna falla de concurrencia permita vender artículos inexistentes.**

- **Criterios de Aceptación:**
  1. **Implementar un `TRIGGER` tipo `BEFORE UPDATE` en PostgreSQL.**
  2. **Lanzar una excepción si `nuevo_stock < 0`.**
  3. **FastAPI debe atrapar esta excepción y retornar HTTP `409 Conflict`.**

Módulo 3: Punto de Venta y Transacciones (Node.js + Firestore)

HU11: Lógica de Carrito de Compras en Memoria (BLoC)

- **Prioridad: Alta | Estimación: 13 Puntos de Historia**
- **Descripción: * Como Trabajador (Vendedor).**
  1. **Quiero agregar productos a un ticket temporal y ver el cálculo de precios.**
  2. **Para procesar el cobro de múltiples artículos.**
- **Criterios de Aceptación:**
  1. **`CarritoBloc` manejará eventos: `AgregarItem`, `RestarItem`, `EliminarItem`.**
  2. **Recálculo instantáneo de `Subtotal` y `Total` en el Drawer lateral.**
  3. **Deshabilitar el botón "+" si la cantidad en el carrito iguala el `stock_actual`.**

HU12: Selección de Método de Pago

- **Prioridad: Media | Estimación: 3 Puntos de Historia**
- **Descripción: * Como Trabajador (Vendedor).**
  1. **Quiero registrar si el cliente pagó en efectivo o con tarjeta.**
  2. **Para cuadrar la caja al final del día.**
- **Criterios de Aceptación:**
  1. **Al presionar "Cobrar", abrir un Modal/Dialog en Flutter para elegir `Efectivo` o `Tarjeta`.**
  2. **Incluir esta variable en el JSON que se enviará al servidor.**

HU13: Orquestación Distribuida de la Venta (Core Transaccional)

- **Prioridad: Crítica | Estimación: 21 Puntos de Historia**
- **Descripción: * Como Sistema.**
  1. **Quiero coordinar los dos microservicios para registrar una venta.**
  2. **Para mantener la sincronía entre el inventario SQL y los tickets NoSQL.**
- **Criterios de Aceptación:**
  1. **Flutter envía el JSON del carrito a Node.js (`/api/ventas/procesar`).**
  2. **Node.js solicita a Python (`/api/productos/descontar`) reducir el stock.**
  3. **Si Python responde `200 OK`, Node.js guarda el ticket inmutable en la colección `ventas` de Firestore con el `vendedor_id`.**

4. Flutter recibe éxito, vacía el `CarritoBloc` y lanza animación Lottie de confirmación.

HU14: Rollback de Transacción (Compensación)

- **Prioridad: Alta | Estimación: 13 Puntos de Historia**
- **Descripción: * Como Sistema.**
  1. **Quiero revertir el descuento de stock si Firestore falla al guardar el ticket.**
  2. **Para evitar pérdidas fantasmas de inventario.**
- **Criterios de Aceptación:**
  1. **Si Node.js falla al escribir en Firebase (ej. timeout), debe atrapar el error.**
  2. **Node.js enviará una petición de compensación a Python (`/api/productos/revertir`) para devolver el stock descontado.**
  3. **Devolver HTTP `500` a Flutter indicando "Venta fallida, intente de nuevo".**

Módulo 4: Auditoría y Dashboard (Consultor/Admin)

HU15: Historial de Ventas del Turno

- **Prioridad: Media | Estimación: 5 Puntos de Historia**
- **Descripción: * Como Trabajador (Vendedor).**
  1. **Quiero ver las ventas que he realizado hoy.**
  2. **Para llevar mi propio control antes de entregar la caja.**
- **Criterios de Aceptación:**
  1. **Pantalla que consulte a Node.js las ventas filtradas por `vendedor_id == mi_id` y fecha actual.**
  2. **Mostrar listado con total y hora.**

HU16: Historial Global de Ventas

- **Prioridad: Media | Estimación: 5 Puntos de Historia**
- **Descripción: * Como Administrador.**
  1. **Quiero visualizar absolutamente todas las transacciones de la tienda.**
  2. **Para auditar el trabajo de los cajeros.**
- **Criterios de Aceptación:**
  1. **Consultar a Firestore la colección `ventas` completa, ordenada descendentemente por fecha.**
  2. **Mostrar nombre del cajero, método de pago y desglose de productos al expandir la tarjeta.**

HU17: Cancelación de Venta (Anulación)

- **Prioridad: Alta | Estimación: 8 Puntos de Historia**

- **Descripción: * Como Administrador.**
    1. **Quiero anular una venta registrada por error o devolución.**
    2. **Para corregir los ingresos y retornar el producto al inventario.**
- **Criterios de Aceptación:**
    1. **En el historial global, botón "Anular Venta" (solo Admin).**
    2. **Node.js cambia el estado del ticket en Firestore a `anulada: true`.**
    3. **Node.js envía a Python la orden de sumar el stock de los productos involucrados en esa venta específica.**

HU18: Dashboard de Métricas en Tiempo Real

- **Prioridad: Alta | Estimación: 13 Puntos de Historia**
- **Descripción: * Como Administrador.**
    1. **Quiero ver gráficas de rendimiento empresarial.**
    2. **Para tomar decisiones sobre qué patinetas comprar para la siguiente semana.**
- **Criterios de Aceptación:**
    1. **Pantalla de Dashboard en Flutter usando `fl_chart`.**
    2. **Mostrar tarjetas de resumen: "Ventas del día", "Ingresos Totales", "Top Vendedor".**
    3. **Consultas optimizadas en Node.js usando agregaciones de Firestore.**

HU19: Generación de Reportes PDF/Excel

- **Prioridad: Alta | Estimación: 8 Puntos de Historia**
- **Descripción: * Como Consultor.**
    1. **Quiero descargar un reporte formal del inventario y las ventas.**
    2. **Para presentarlo en las reuniones de dirección.**
- **Criterios de Aceptación:**
    1. **Endpoint en Python que utilice `ReportLab` o `Pandas`.**
    2. **El archivo incluirá logotipo de SkateShopPachuca, fecha de emisión y tabla de datos formateada.**
    3. **Descarga nativa en el dispositivo usando `path_provider` en Flutter.**

Módulo 5: Tolerancia a Fallos y UX Avanzada

HU20: Caché Local de Catálogo (Offline First)

- **Prioridad: Media | Estimación: 8 Puntos de Historia**
- **Descripción: * Como Trabajador (Vendedor).**
    1. **Quiero poder ver los productos aunque haya micro-cortes de internet.**
    2. **Para no detener la atención al cliente si la red falla temporalmente.**
- **Criterios de Aceptación:**
    1. **Tras una consulta exitosa a Python, Flutter guardará el array JSON en una base de datos local (ej. paquete `Hive` o `Isar`).**

2. **Al abrir la app, cargar primero los datos locales (`CachePolicy`) mientras se refresca el fondo.**

HU21: Manejo de Excepciones de Red

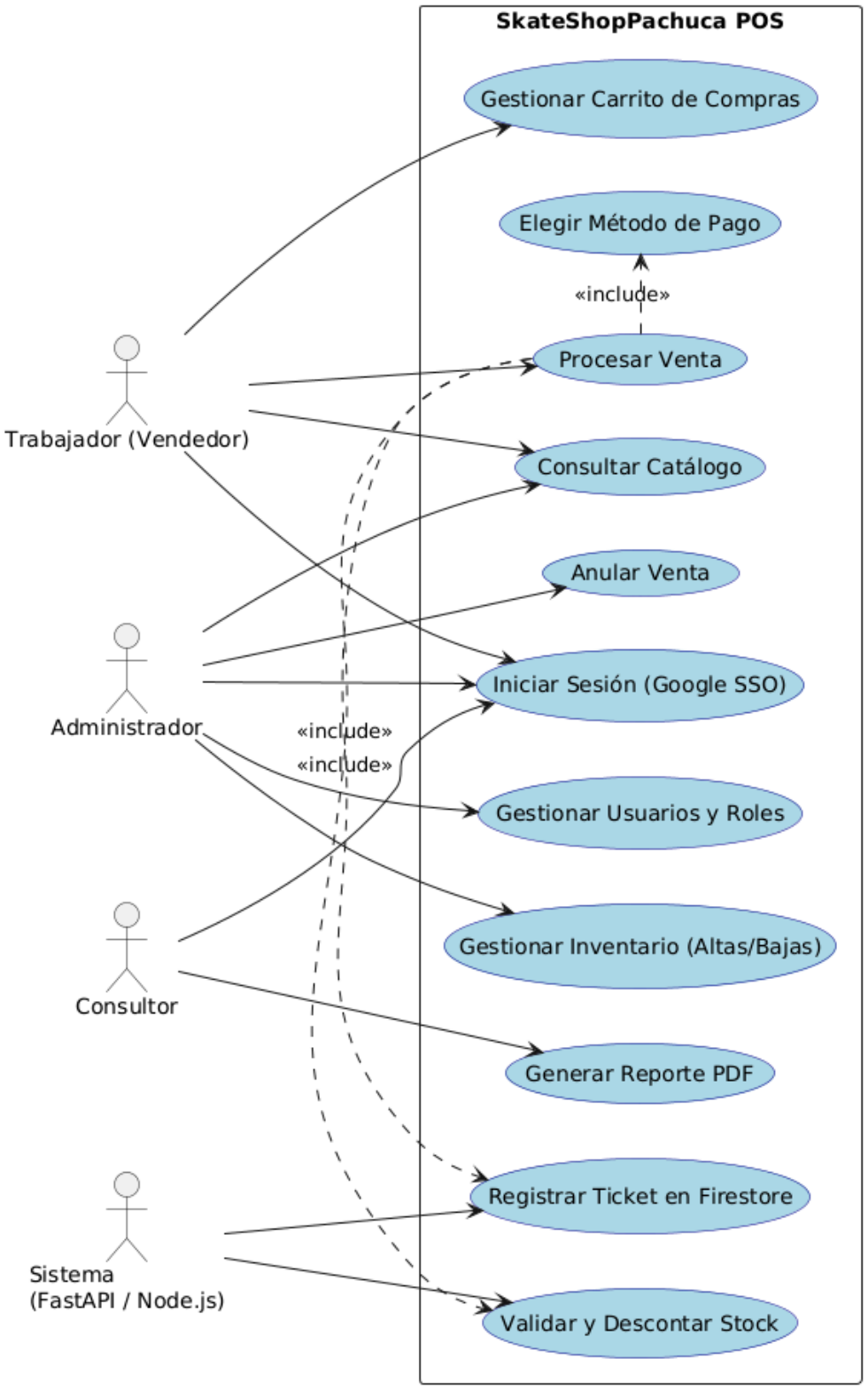
- **Prioridad: Alta | Estimación: 5 Puntos de Historia**
- **Descripción: * Como Sistema.**
  1. **Quiero capturar los errores de servidor (Ej. Error 500) y de desconexión.**
  2. **Para evitar que la aplicación se congele (Crash) y guiar al usuario.**
- **Criterios de Aceptación:**
  1. **Interceptar `SocketException` en los repositorios de Flutter.**
  2. **Emitir estado `Error` en BLoC.**
  3. **Renderizar una vista amigable de "No hay conexión" con un botón de "Reintentar".**

HU22: Autenticación Backend-to-Backend (Seguridad API)

- **Prioridad: Alta | Estimación: 5 Puntos de Historia**
- **Descripción: * Como Arquitecto de Seguridad.**
  1. **Quiero asegurar que nadie fuera de nuestro ecosistema pueda alterar el inventario.**
  2. **Para prevenir ataques directos al servicio de Python.**
- **Criterios de Aceptación:**
  1. **FastAPI requerirá un `X-API-KEY` en los headers de los endpoints críticos (`/descontar`).**
  2. **Node.js almacenará esta llave como variable de entorno (`.env`) y la inyectará al hacer el *fetch* hacia Python.**

7.2 Especificación de Casos de Uso (UML)

Para complementar el Product Backlog, se detallan los Casos de Uso Core del sistema, definiendo la interacción exacta entre los Actores (Vendedor, Administrador, Sistema) y los microservicios.



CU-01: Procesar Venta en Mostrador

- **Actor Principal: Trabajador (Vendedor).**
- **Actores Secundarios: Backend Node.js, Backend FastAPI.**
- **Precondiciones: El Vendedor debe estar autenticado en el sistema y el carrito debe tener al menos un producto.**
- **Flujo Principal:**
  - **El Vendedor revisa el carrito y presiona "Confirmar Venta".**
  - **El sistema (Flutter) solicita elegir el método de pago.**
  - **El Vendedor selecciona "Efectivo" o "Tarjeta" y confirma.**
  - **El sistema envía el payload a Node.js (`/api/ventas/procesar`).**
  - **Node.js se comunica con FastAPI para descontar el inventario en PostgreSQL.**
  - **FastAPI descuenta el stock y responde `200 OK`.**
  - **Node.js registra el ticket en Firestore y responde `201 Created` al cliente.**
  - **La interfaz muestra mensaje de éxito y vacía el carrito.**
- **Flujos Alternativos:**
  - ***5a. Stock Insuficiente:* FastAPI detecta que no hay inventario físico y aborta mediante el *Trigger* de PostgreSQL. Responde `409 Conflict`. Node.js reenvía el error a Flutter. El sistema avisa al Vendedor: "Stock insuficiente para procesar la venta".**
- **Postcondiciones: El inventario físico se reduce y se genera un ticket inmutable en la base de datos NoSQL.**

CU-02: Gestión de Inventario (Entradas/Salidas)

- **Actor Principal: Administrador.**
- **Actores Secundarios: Backend FastAPI.**
- **Precondiciones: El Administrador debe tener una sesión activa con token JWT válido.**
- **Flujo Principal:**
  - **El Administrador navega a la vista de "Gestión de Inventario".**
  - **Selecciona un producto y elige la opción "Registrar Entrada".**
  - **Ingresa la cantidad recibida por el proveedor y el motivo ("Compra a Vans").**
  - **El sistema envía la petición a FastAPI.**
  - **FastAPI inserta el registro en la tabla `movimientos_inventario`.**
  - **El Trigger de PostgreSQL actualiza el `stock_actual` sumando la nueva cantidad.**
  - **El sistema muestra "Inventario actualizado exitosamente".**
- **Flujos Alternativos:**
  - ***3a. Datos Inválidos:* El Administrador ingresa una cantidad negativa. El frontend bloquea el botón de envío y muestra "La cantidad debe ser mayor a cero".**
- **Postcondiciones: El catálogo refleja el nuevo stock disponible para los cajeros en tiempo real.**

CU-03: Generación de Reporte Directivo

- Actor Principal: Consultor / Dueño.
- Actores Secundarios: Backend FastAPI.
- Precondiciones: El usuario tiene rol de Consultor o Administrador.
- Flujo Principal:
    1. El Consultor accede al "Dashboard de Reportes".
    2. Selecciona el rango de fechas (Ej. Mes actual) y presiona "Generar PDF".
    3. El sistema (Flutter) envía la petición GET a FastAPI con los parámetros de fecha.
    4. FastAPI compila los datos de ventas usando Pandas, dibuja el PDF con ReportLab y retorna el stream de bytes.
    5. El sistema descarga el archivo y abre el visor de PDF nativo del dispositivo.
- Postcondiciones: Un archivo PDF validado es guardado en el almacenamiento local del dispositivo del Consultor.

---

8. Estrategia Integral de Pruebas (Estándar IEEE 829 Avanzado)

Para garantizar la fiabilidad del sistema POS bajo condiciones de estrés, se implementó una estrategia de pruebas en tres niveles, abarcando desde el código fuente hasta la interfaz visual.

8.1 Pruebas Unitarias (Caja Blanca)

Se enfocan en validar la lógica interna de los componentes sin depender de bases de datos o red.

- **Herramientas: `flutter_test`, `bloc_test` (Dart), `pytest` (Python).**
- **Ejemplo de Prueba (Flutter BLoC):**
    - **Objetivo: Validar que el `CarritoBloc` calcula correctamente el total.**
    - **Acción: Emitir evento `AgregarProducto` con un producto de $500 y cantidad 2.**
    - **Aserción: Se espera que el estado `CarritoActualizado` tenga una variable `total == 1000`.**

8.2 Pruebas de Integración y API (Caja Negra)

Se enfocan en validar las entradas y salidas de los servicios REST sin conocer su código interno, simulando peticiones HTTP.

- **Herramientas: Postman, Thunder Client.** ***(Nota: Agregar capturas de pantalla de Postman aquí).***

Caso de Prueba (Postman) - Orquestación de Stock:

- **ID: CP-INT-01**
- **Precondición: El producto ID 102 tiene 1 unidad en stock en PostgreSQL.**
- **Acción: Enviar un POST a `/api/ventas/procesar` (Node.js) intentando comprar 5 unidades.**
- **Comportamiento Esperado: Node.js hace el puente con Python. Python rechaza. Node.js devuelve `HTTP 409 Conflict` con un JSON `{"error": "Stock insuficiente para el ID 102"}`.**

8.3 Pruebas de Sistema y UI (End-to-End)

Se validan los flujos completos desde la perspectiva del usuario final en el dispositivo físico o emulador.

Caso de Prueba UI - Resiliencia de Red:

- **ID: CP-E2E-01**
- **Objetivo: Evitar ventas fantasma por caídas de internet.**
- **Pasos:**
  1. **El vendedor agrega 3 productos al carrito.**
  2. ***Desactivar el Wi-Fi del dispositivo móvil.***
  3. **Presionar el botón "Confirmar Venta".**
- **Resultado Esperado: La app no debe congelarse (Crash). El bloque `try/catch` del repositorio debe atrapar la `SocketException` y mostrar un Snackbar rojo: "Error de red. Verifique su conexión."**

8.4 Pruebas de Seguridad en Base de Datos (SQL Testing)

- **Objetivo: Validar que los *Triggers* y *Constraints* de PostgreSQL son impenetrables.**
- **Acción: Ejecutar directamente en pgAdmin 4: `UPDATE productos SET stock_actual = -5 WHERE id = 101;`**
- **Resultado Esperado: El motor de PostgreSQL detiene la ejecución y arroja la excepción personalizada programada por el equipo de Backend.**

9. Métricas de Control Ágil y Definition of Done (DoD)

Para garantizar la calidad empresarial y evitar la acumulación de deuda técnica durante el desarrollo del Punto de Venta, el equipo aplica estrictamente las siguientes métricas de evaluación continua:

1. **Gráfica de Quemado (Burndown Chart): Seguimiento en Trello del progreso de los Puntos de Historia (esfuerzo) comprometidos en cada Sprint contra los completados diariamente.**

2. **Deuda Técnica Cero: Es una regla estricta de control de versiones. El comando `flutter analyze` debe retornar siempre "No issues found!". No se aceptan ramas en GitHub con advertencias o variables sin utilizar.**
3. **Cobertura de Excepciones: Todas las llamadas a las APIs de Python y Node.js deben estar envueltas en bloques `try/catch` para evitar cierres inesperados de la aplicación (Crashes).**

9.1 Definition of Done (DoD)

Para que Samuel, Karel o Daniel puedan mover una tarjeta de Historia de Usuario a la columna de "Terminado" (Done), el incremento de software debe cumplir obligatoriamente esta lista de verificación:

- **[x] El código está fusionado en la rama `main` de GitHub mediante un Pull Request sin conflictos.**
- **[x] Cumple al 100% con los Criterios de Aceptación definidos en la sección 7.**
- **[x] El código fuente cuenta con documentación automática (`///` DartDoc).**
- **[x] Ha pasado con éxito al menos un Caso de Prueba documentado bajo el estándar IEEE 829.**

---

10. Enlaces y Repositorios Oficiales del Proyecto

Para efectos de evaluación, trazabilidad y control de versiones, se adjuntan los accesos oficiales:

- **Repositorio de Código Fuente (GitHub): https://github.com/ADanicode/SkateShopPachuca**
- **Tablero Ágil Scrum (Trello): [Enlace al tablero oficial del equipo]**
- **Directorio de Documentación (Google Drive): https://drive.google.com/drive/folders/1gysIV7s-jfFX_GMZi1Way-62bSYimr79?usp=drive_link**
- **Grupo Oficial de Comunicación Interna (WhatsApp): https://chat.whatsapp.com/ChiOxt7S5o0FaW0KpD9R3x**

---

11. Bibliografía y Referencias Técnicas

La arquitectura e implementación técnica se fundamentan en la documentación oficial de las siguientes tecnologías:

1. **Flutter BLoC & Clean Architecture: Felangel. (2024). *Bloc Library Documentation*. Recuperado de: https://bloclibrary.dev/**
2. **FastAPI Framework: Ramírez, S. (2024). *FastAPI Documentation*. Recuperado de: https://fastapi.tiangolo.com/**

3. **PostgreSQL Documentation (Triggers & Integrity): PostgreSQL Global Development Group. (2024). *PostgreSQL 18 Documentation*. Recuperado de: https://www.postgresql.org/docs/**
4. **Firebase Admin SDK & Firestore: Google. (2024). *Firebase Documentation para Node.js y Flutter*. Recuperado de: https://firebase.google.com/docs**
5. **Ingeniería de Software (Estándares): IEEE Standards Association. *IEEE 830-1998 (SRS)* e *IEEE 829-2008 (Test Documentation)*.**